

IN2354 Kinect Fusion - Final Report

Mubashir Hanif, William Koch, Christoph Pillmayer

February 23, 2021

1 Abstract

In this report we first introduce the kinect fusion approach and why it is interesting to implement. Afterwards an overview over the method is given. The main part task about different implementation details as well as the GUI and marching cubes implementation. Finally we briefly go over experimental results and give a conclusion.

2 Introduction

Implementing Kinect Fusion [1] is an interesting topic for several reasons. It uses commodity hardware, therefore the group of potential users is large. Its technical approach is, by design, well suited for computation using GPU and therefore capable of producing results in real-time. The output is a dense reconstruction that can be useful in many applications.

One example that comes to mind first surely is AR. Here, the dense reconstruction enables detailed occlusions and interactions with the surface, which greatly enhances the user experience. Another use case one can think of are rescue missions. The ability to generate detailed maps of an environment, which might be hard to navigate, can save lives.

3 Related Work

The overall process, as described in [1], is split into four parts, as seen in figure 2. In a first step, we process the depth map from the RGB-D camera to compute vertex and normal maps. Using these, we can then compute a global Truncated Signed Distance Function (TSDF), where we also associate weights to each value. We can then predict the surface with the global SDF using ray-casting. Information about the reconstructed surface is then used to align future sensor frames to the existing model.

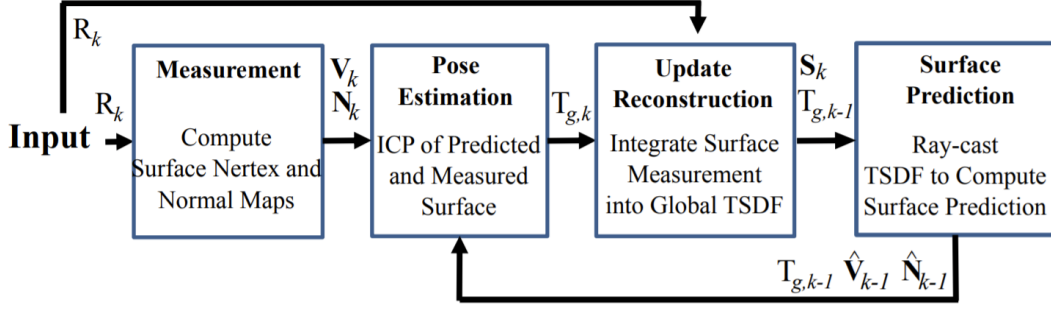


Figure 1: Overall system workflow. [1]

4 Method

The implementation is centered around an event loop that is used for rendering different views of the current TSDF as well as other debugging elements such as vertex and normal maps. To make it easier to implement drawing of multiple meshes, we use a simple class that contains the vertex and element buffer and exposes methods for setting the buffers, drawing mode, vertex buffer layout and also drawing itself. We use GLFW as a light weight window manager and GLEW for the OpenGL bindings. The GUI makes use of the *imgui* [3] library. The kinect fusion method itself is implemented in a single class `pipeline` which stores various parameters and buffers used during processing.

On startup, memory for the TSDF is allocated on the GPU and buffers for the vertex and normal maps are allocated on the host as well as the GPU. Afterwards the method `process_frame(float* depth, uint* rgbx)` of the pipeline is used to input new data coming from a dataset or a kinect sensor.

4.1 Surface Measurement

In this stage of the process, we generate the vertex and normal maps $V_{k,c}, N_{k,c}$ in camera space. Each CUDA thread computes the vertex and normal for a specific pixel from the depth map.

While the original paper [1] implemented multi-level pyramids to generate vertex and normal maps of different resolutions, we only worked with the base layer (full resolution) in practice.

We implemented a bilateral filter for the raw depth map. However, this did not noticeably improve our results while at the same time slowing down our pipeline from approximately 17 FPS to 10 FPS (not using shared memory in CUDA).

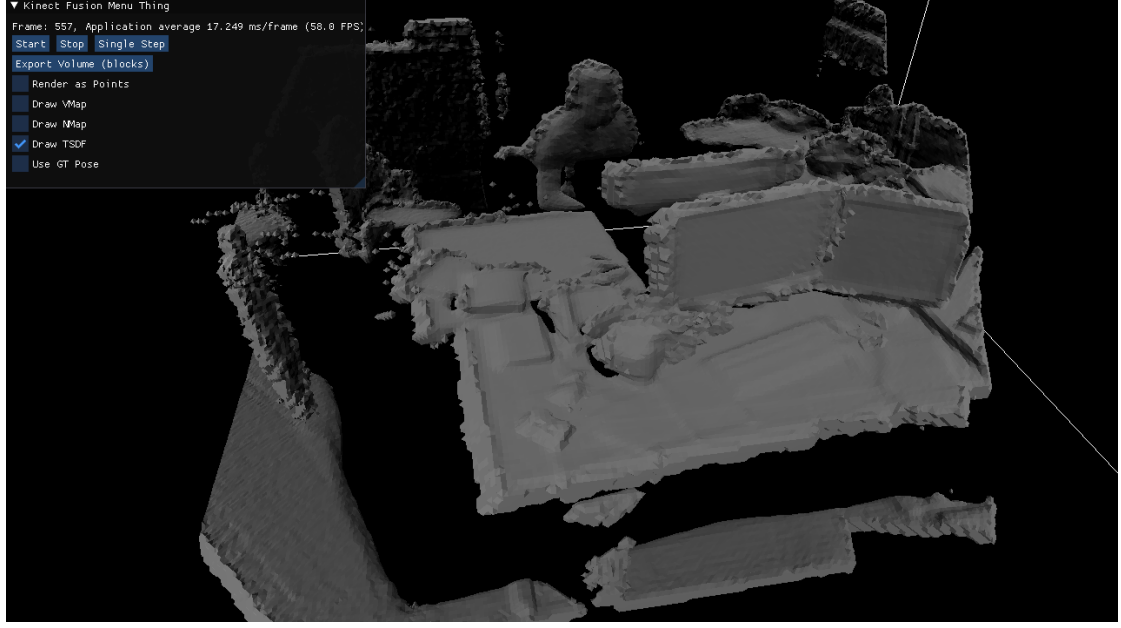


Figure 2: Appearance(global TSDF) of our implementation with the freiburg1_xyz dataset [2]

4.2 Pose Estimation

Pose estimation is the 2nd step in the pipeline. Pose estimation runs from the second iteration of the algorithm. It has 5 arguments, vertices and normals from surface measurement 4.1 on current frame, pose from the previous frame and the vertices and normals from the surface prediction 4.4. We start by copying vertices and normals on the GPU memory. Each cuda thread then runs projective data association kernel [4] between the previous frame and the current frame. If the projection of vertex $V_k(u)$ from the current frame into the previous frame results in a valid point then $V_k(u)$, $V_{k-1}(u)$ and $N_{k-1}(u)$ are added to the correspondence list. We then take the correspondence list and construct the linear system to estimate the pose. To construct the linear system $Ax = b$ where $A \in R^{N \times 6}$ matrix, $b \in R^N$ and $x = (\alpha, \beta, \gamma, t_x, t_y, t_z)^T$, here $N = Width \times Height$. The k^{th} rows of the constraint matrix A is computed as $A_k = V_k \times N_{k-1}, N_{k-1}$ and the k^{th} element of image b is $b_k = N_{k-1} \cdot (V_{k-1} - V_k)$.

To solve this linear system, unlike [1] we computed $A^T A \in R^{6 \times 6}$ and $A^T b$ on the GPU. Here $A^T A$ is a SPD matrix and can be inverted very efficiently on the CPU using Cholesky Decomposition. Finally the transformation matrix T_k^g is computed from the solution vector x using the definition from [5].

4.3 Fusion

In the update reconstruction step, the raw depth measurement is fused into the global TSDF using the estimated pose from Section 4.2 by projecting each voxel into camera

space. This is used to obtain the observed depth value $F_{R_k}(p)$ for the voxel $p \in R^3$, which allows us to update the TSDF in a weighted running average. We assign a constant weight of 1 to the new values.

4.4 Surface Prediction

For the ICP we compute the global vertex map $V_{k,g}$ from the TSDF. We compute the vertices by performing a raycast from the current camera pose through our voxel grid. The TSDF values of the individual voxels allow us to identify when we encounter a zero crossing. We use trilinear interpolation to estimate the TSDF values, which noticeably improved our camera tracking.

4.5 Rendering

To be able to export and render the TSDF in the OpenGL debugging window, marching cubes is used to extract the implicit surface from the TSDF. There are many implementation variants that can be used and generally represent a tradeoff between implementation complexity and speed. Here, a simple approach is used as it is not the main focus of the project. The used method consists of three steps. First, every voxel is checked for its state with regards to the amount of vertices it will produce. This value is stored in a buffer. In the next step the buffer is processed using a parallel exclusive scan. For an exemplary input $i = [1, 2, 1]$ the output is $o = [0, 1, 3]$. This basically converts the vertex amounts per voxel into an offset per voxel that is used later to be able to index into the output buffers. In the third step we simply go over the volume again and generate the output vertices, normals and triangle indices. The tables for marching cubes we use are taken from [6].

We also implement an alternative technique for rendering the TSDF, where we simply triangulate an output of the surface prediction function that is called with a suitable pose to generate a nice view on the volume. Marching cubes is still used for exporting the volume contents in this case.

The viewport shows the extracted surface with a simple phong shading as well as additional elements such as the current camera pose.

5 Benchmarking

Figure 3 shows that the performance bottleneck is the fusion step, accounting for more than 70% of all GPU usage. In the original paper [1], an update in the update reconstruction method (fusion) took 2 ms on a 512^3 grid, whereas our implementation takes 55 ms. This is an indication that our CUDA code still offers room for improvement.

Volume Size	FPS		GPU Memory Usage (MB)
	w/ visualization	w/o visualization	
256 ³	16	23	310
384 ³	10	16	630
512 ³	10	6	1125

Table 1: Measurements performed on the freiburg_xyz dataset [2]. Ryzen 5600X with 16GB main memory, GTX 1070

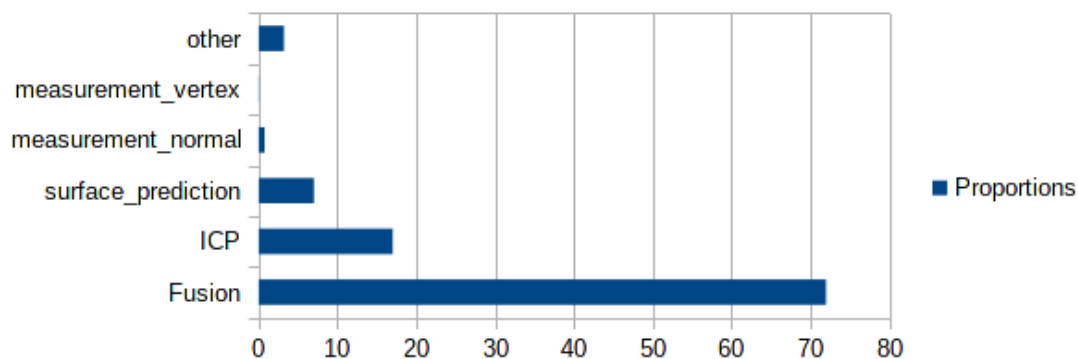


Figure 3: Performance breakdown by kernel (in %, 512³ grid, no visualization)

6 Conclusion

The overall pipeline is able to track the camera movement successfully in the freiburg_xyz dataset. In the freiburg_rpy and freiburg_360 datasets, where there is a lots more rotation, tracking is lost a lot faster, after about 25 to 30 frames.

Using marching cubes in real-time for rendering was not successful, a more complex implementation as described in [7] could be used in conjunction with the OpenGL to CUDA interface to directly write the pixel buffer from CUDA. This approach would save significant computation and memory transfer time.

Overall the implementation would probably have been easier by relying on a library like OpenCV which also has GPU support and of course comes with lots of built in functions already. As none of us had worked with CUDA before, we spent much more time understanding CUDA memory and debugging than we expected, but it truly was a valuable learning experience.

References

- [1] D. Kim et al. “KinectFusion: Real-time dense surface mapping and tracking”. In: *2011 IEEE International Symposium on Mixed and Augmented Reality*. Los Alamitos, CA, USA: IEEE Computer Society, Oct. 2011, pp. 127–136. DOI: 10.1109/ISMAR.2011.6092378. URL: <https://doi.ieeecomputersociety.org/10.1109/ISMAR.2011.6092378>.
- [2] J. Sturm et al. “A Benchmark for the Evaluation of RGB-D SLAM Systems”. In: *Proc. of the International Conference on Intelligent Robot Systems (IROS)*. Oct. 2012.
- [3] Cornut Omar and Contributors. *Dear ImGui*. <https://github.com/ocornut/imgui>. 2020.
- [4] G. Blais and M. D. Levine. “Registering multiview range data to create 3D computer objects”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 17.8 (1995), pp. 820–824. DOI: 10.1109/34.400574.
- [5] Kok-lim Low. *Linear least-squares optimization for point-to-plane ICP surface registration*. Tech. rep. 2004.
- [6] Diller Christian. *Kinect Fusion Lib*. <https://github.com/chrdiller/KinectFusionLib>. 2016.
- [7] Hubert Nguyen. *Gpu Gems 3*. First. Addison-Wesley Professional, 2007. ISBN: 9780321545428.